

MongoDB Master Data Engineer & Senior Architect

6-Month Exhaustive Architecture, Aggregation, Security, and DataOps Program

Distributed Systems, WiredTiger Internals, Vector Search, and Kubernetes DataOps

Abstract

This 24-week program is designed to transform you into a Master Data Engineer and Senior Architect in MongoDB. It covers WiredTiger storage engine internals, specialized document schema design patterns, relational database migrations, multi-region sharded cluster mechanics, advanced aggregation pipelines, Atlas Vector Search, Queryable Encryption (QE), Atlas Stream Processing, and Kubernetes GitOps automation. A tracking timetable is included at the end.

Contents

1 Milestone 1: WiredTiger Internals, Schema Patterns & Migration (Month 1)	2
2 Milestone 2: High Availability, Sharding Mechanics & Scalability (Month 2)	7
3 Milestone 3: Aggregation Framework, Indexes & Performance Tuning (Month 3)	10
4 Milestone 4: Atlas Ecosystem, Search, Vector Search & Streaming (Month 4)	15
5 Milestone 5: Enterprise Security, Governance, Encryption & Compliance (Month 5)	21
6 Milestone 6: DataOps, Automation, Kubernetes & Certification (Month 6)	26
7 Appendix A: Glossary	31
8 Appendix B: Assessment & Grading Model	32
9 Appendix C: Interview Preparation	33
10 Appendix D: Self-Check Answer Key	33
11 Appendix E: Datasets & Project Data	37
12 Appendix F: Portfolio & Presentation Guide	38
13 6-Month Progress Tracking Timetable	40

Prerequisites & How to Use This Syllabus

- **Prerequisite skills:** intermediate Python, solid SQL fundamentals, Git workflows, Linux/CLI basics (or Windows PowerShell equivalents), and Docker fundamentals for running local `mongod/mongos` deployments.
- **Weekly cadence:** approximately 10–12 hours per week — Monday to Wednesday for theory and internals, Thursday for the hands-on project, Friday for the architecture use case.
- **Deliverable expectation:** every capstone must produce a git repository, an architecture diagram, a cost estimate, and a security review checklist (per the Acceptance Criteria boxes that follow each capstone).
- **Assessment:** the 24-week tracking timetable at the end of this document doubles as the assessment checklist — tick each week only when its project and resources are complete.

Environment setup checklist:

- Install Python 3.x, Git, Docker Desktop, and VS Code (with the MongoDB for VS Code extension).
- Install the platform CLIs: `mongosh`, local `mongod/mongos` binaries, the Atlas CLI (`atlas`), and Terraform.
- Create a free-tier account: an Atlas M0 sandbox cluster with the sample datasets loaded (see Appendix E).
- Clone or create a course repository structured as `data/`, `src/`, `tests/`, `infra/`, `docs/`.
- Configure credentials via environment variables (`MONGODB_URI`, Atlas API keys) — never hardcode secrets in code or commits.

1 Milestone 1: WiredTiger Internals, Schema Patterns & Migration (Month 1)

Objective: Master the WiredTiger engine, document data modeling patterns, and relational-to-document migration.

Learning outcomes: tune WiredTiger cache and interpret eviction metrics, apply document schema design patterns, enforce `$jsonSchema` validation, execute a zero-downtime RDBMS migration with CDC.

Week 1: MongoDB Core Architecture & WiredTiger Storage Engine

Outcomes: size the WiredTiger cache against the OS page cache, interpret eviction metrics in `db.serverStatus().wiredTiger`, explain journal vs. checkpoint durability, detect checkpoint I/O spikes.

- **Monday:** MongoDB process architecture: `mongod` daemon, thread pools, and memory management.
- **Tuesday:** WiredTiger Internals: B-Trees, eviction policies, cache size configuration, and page allocation.
- **Wednesday:** Write Path: In-memory cache, Journaling, checkpointing (60-second default), and crash recovery.

- **Thursday: Hands-on Project:** Configure a local WiredTiger deployment. Modify `storage.wiredTiger`. generate 10 million inserts using `mgeneratejs`, and analyze eviction metrics via `db.serverStatus().wiredTiger`.
- **Friday: Real-World Architecture:** Architect a high-throughput financial ledger engine, balancing WiredTiger cache allocation against OS page cache to eliminate checkpoint I/O spikes.

Resources: MongoDB WiredTiger documentation (<https://www.mongodb.com/docs/manual/core/wiredtiger/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>)

Self-check:

- What is WiredTiger's default checkpoint interval?
- What does the journal protect that checkpoints alone do not?
- Which `serverStatus` section exposes cache eviction metrics?
- Why should the WiredTiger cache not consume all available RAM?
- What production symptom indicates checkpoint I/O spikes?

Week 2: Advanced Schema Design Patterns & Analytics Modeling

Outcomes: apply Attribute, Bucket, Subset, and Computed patterns to concrete models, build an SCD Type 2 history collection from a Change Stream, expose a star-schema-style view via the Atlas SQL Interface.

- **Monday:** Core Design Patterns: Polymorphic, Attribute, Bucket, and Outlier Patterns.
- **Tuesday:** Operational Patterns: Computed, Subset, Extended Reference, and Schema Versioning Patterns. **Modeling for analytics:** star-schema-style collections and read-optimized views for the Atlas SQL Interface / BI Connector, grain declaration for aggregation pipelines, and SCD Type 1 (overwrite) vs. Type 2 (versioned history) in documents.
- **Wednesday:** Data Lifecycle Patterns: Time-To-Live (TTL), Pre-allocation, and Archival Patterns.
- **Thursday: Hands-on Project:** Refactor a monolithic e-commerce product catalog. Implement the Attribute Pattern for dynamic specs and the Subset Pattern to optimize working set memory. Then build an SCD Type 2 `dim_customers` history collection driven by a Change Stream on the operational `customers` collection, and expose a star-schema-style reporting view (fact orders + conformed dimensions) through the Atlas SQL Interface.

```
from datetime import datetime, timezone
from pymongo import MongoClient

db = MongoClient("mongodb://localhost:27017").analytics

def apply_scd2(dim: dict) -> None:
    """SCD Type 2: expire the current version, insert the new one."""
    now = datetime.now(timezone.utc)
    db.dim_customers.update_one(
        {"customer_id": dim["customer_id"], "valid_to": None},
        {"$set": {"valid_to": now}})
    db.dim_customers.insert_one(
        {**dim, "valid_from": now, "valid_to": None})

# History is maintained by a Change Stream on the source collection.
ops = db.oltp # operational database
with ops.customers.watch(
```

```
[{"$match": {"operationType": {"$in": ["insert", "update"]}}}] as cs:
for change in cs:
    apply_scd2(change["fullDocument"])
```

- **Friday: Real-World Architecture:** Design an IoT device metrics schema processing 100,000 events/sec using the Bucket Pattern to reduce document count and metadata overhead by 95%.

Resources: MongoDB Schema Design Patterns blog series (<https://www.mongodb.com/blog/post/building-with-patterns-a-summary>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>). **Self-check:**

- Which pattern fits products with highly variable attributes?
- How does the Bucket Pattern reduce document count and index size?
- When should you apply the Subset Pattern?
- What does the Computed Pattern move from read time to write time?
- Contrast SCD Type 1 vs. Type 2 in a document collection.

Week 3: Modeling Relationships, Anti-Patterns & Data Contracts

Outcomes: choose embedding vs. referencing per relationship, identify unbounded-array and 16MB-bloat anti-patterns, author a strict \$jsonSchema validator, classify schema changes as breaking vs. non-breaking.

- **Monday:** Embedding vs. Referencing: 1:1, 1:N (few), 1:N (many), and N:M relationship choices.
- **Tuesday:** Document Anti-Patterns: Unbounded Arrays, Bloated Documents (> 16MB), and Massive Collections.
- **Wednesday:** JSON Schema Validation as Data Contracts: \$jsonSchema as a versioned contract — schema versioning pattern, breaking (renaming/removing required fields, narrowing types) vs. non-breaking changes (adding optional fields), producer-vs-consumer ownership, and CI validation of representative documents before deploy.
- **Thursday: Hands-on Project:** Write a JSON Schema validator for an enterprise HR system. Restrict document sizes, mandate required fields, and use \$jsonSchema in createCollection.

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017")
db = client.hr

employee_schema = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["employee_id", "name", "department", "hire_date"],
        "properties": {
            "employee_id": {"bsonType": "string", "pattern": "^EMP-[0-9]{6}$"},
            "name": {"bsonType": "string", "minLength": 1},
            "department": {"enum": ["Engineering", "Sales", "HR", "Finance"]},
            "salary": {"bsonType": ["int", "double"], "minimum": 0},
            "hire_date": {"bsonType": "date"},
        }
    }
}
```

```
db.create_collection("employees", validator=employee_schema,
                    validationLevel="strict", validationAction="error")
```

- **Friday: Real-World Architecture:** Analyze a legacy system failing due to BSONObj size exceeds 16MB errors on user activity logs, re-architecting it using parent-referencing and capped collections.

Resources: MongoDB \$jsonSchema validation docs (<https://www.mongodb.com/docs/manual/core/schema-validation/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>). **Self-check:**

- When do you embed a 1:N relationship instead of referencing it?
- Why are unbounded arrays considered an anti-pattern?
- Give one breaking and one non-breaking schema change.
- What does validationAction: "warn" do?
- Who owns a data contract, and where is it enforced?

Production Note: Schema-drift contract testing in CI: before deploying application changes, run a CI step that validates a set of representative documents (sampled from production-like staging data) against the collection's \$jsonSchema validator via a dry-run insert into a scratch collection. This catches producer-side schema drift before strict validators start rejecting writes in production.

Week 4: Relational to Document Migration Strategy

Outcomes: map a 3NF schema into document collections, configure Relational Migrator initial sync and CDC, design a dual-write plus shadow-read cutover with rollback, verify row-count parity.

- **Monday:** Analyzing RDBMS schemas: Identifying Third Normal Form (3NF) tables to merge into single documents.
- **Tuesday:** MongoDB Relational Migrator: Mapping, transformation rules, and CDC (Change Data Capture) setup.
- **Wednesday:** Zero-Downtime Migration Patterns: Dual-writes, shadow reads, and CDC replication pipelines.
- **Thursday: Hands-on Project:** Set up MongoDB Relational Migrator. Map a 12-table PostgreSQL e-commerce schema into 3 MongoDB collections, execute initial sync, and verify CDC.
- **Friday: Real-World Architecture:** Design a zero-downtime migration strategy for a 15TB PostgreSQL database to MongoDB Atlas, guaranteeing instant rollback capability and zero data loss.

Resources: MongoDB Relational Migrator docs (<https://www.mongodb.com/docs/relational-migrator/>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>). **Self-check:**

- What are dual-writes, and what risk do they introduce?
- Why is CDC required after the initial sync?

- What do shadow reads validate before cutover?
- Why must migration writes be idempotent?
- What makes a migration instantly rollback-able?

Production Note: *Dual-write migrations are a classic source of divergence: make every write idempotent (natural keys + upserts), log mismatches from shadow-read comparisons to a dead-letter collection, and rehearse the rollback path before cutover day.*

CAPSTONE PROJECT - Milestone 1: Relational Monolith Modernization Engine. Design a complete migration architecture. Convert an open-source 15-table SQL database schema into an optimized document model utilizing Attribute, Bucket, and Subset patterns. Deploy JSON Schema validation, execute continuous CDC sync via Relational Migrator, and run performance benchmarks proving a 5x query speedup.

Acceptance Criteria

- End-to-end migration
- Architecture and tooling
- At least 5x query speedup
- Monitoring and alerting
- Monthly documentation
- Security audit

Milestone 1 Capstone: Reference Solution Sketch

- **Architecture:** PostgreSQL source → Relational Migrator (initial sync + CDC) → Atlas replica set with \$jsonSchema validators — chosen because managed CDC eliminates hand-rolled oplog tailing.
- Model the 15 tables into 3 collections: `customers` (embedded addresses), `orders` (embedded items, Subset Pattern for hot fields), `products` (Attribute Pattern for variable specs).
- Write the target \$jsonSchema validators first; the schema *is* the migration contract.
- Run initial sync, then CDC; build a parity checker comparing row counts and checksums per table.
- Run shadow reads in parallel for 48 hours, logging divergences to a dead-letter collection before cutover.

```
# Parity check: row counts + hash of sorted natural keys
def parity(pg_rows, mongo_docs) -> bool:
    a = sorted(r["id"] for r in pg_rows)
    b = sorted(d["legacy_id"] for d in mongo_docs)
    return len(a) == len(b) and hash(tuple(a)) == hash(tuple(b))
```

- **Pitfall:** dual-writes diverge on partial failure — make every write idempotent via upserts on natural keys.
- **Pitfall:** cutting over before CDC lag is zero loses the last transactions — gate cutover on lag < 1s.
- **Pitfall:** 1:1 table mapping yields a slow document model — merge 3NF tables into aggregates deliberately.

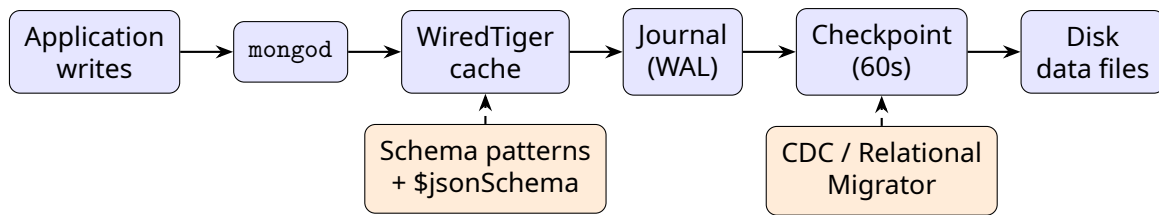


Figure 1: Milestone 1 reference architecture: WiredTiger write path from application to durable checkpoint, with schema validation and migration side processes.

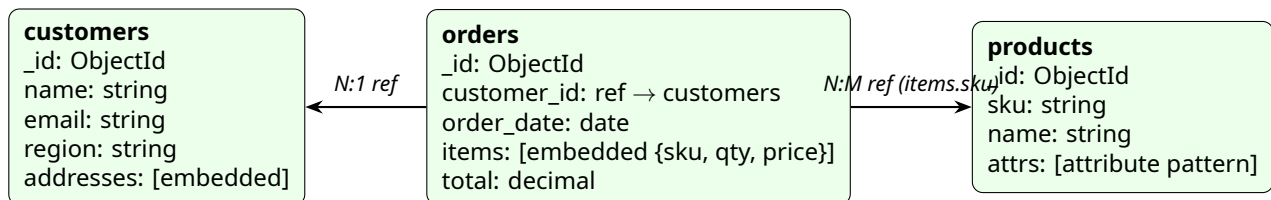


Figure 2: Capstone data model: Relational Monolith Modernization — embedded arrays for addresses and order items, references for customer/product lookups.

2 Milestone 2: High Availability, Sharding Mechanics & Scalability (Month 2)

Objective: Master Consensus protocols, distributed sharding internals, and global data placement.

Learning outcomes: deploy and break replica sets safely, choose write/read concerns for durability targets, design shard keys and refine them live, implement zone sharding for data-residency compliance.

Week 5: Replica Set Internals & Consensus Protocol

Outcomes: explain Raft-like elections and arbiter risks, size the oplog, select write/read concerns for a durability target, trigger and observe a controlled failover under `w:majority`.

- **Monday:** Raft-like Consensus: Elections, Priority, Votes, Heartbeats, and Arbiter risks.
- **Tuesday:** The Oplog (Operations Log): Oplog sizing, idempotent updates, and replication lag debugging.
- **Wednesday:** Write Concerns (`w:1`, `w:majority`, `j:true`) and Read Concerns (`local`, `majority`, `linearizable`).
- **Thursday: Hands-on Project:** Deploy a 3-node Replica Set locally. Force primary elections with `rs.stepDown()` and simulate node loss with `db.shutdownServer()` on a secondary, observing failover behavior and write availability under `w:majority`. On Windows, the firewall-based partition equivalent is `netsh advfirewall` (block the mongod port inbound/outbound); for cloud deployments, toggle entries in the Atlas Network Access List to cut connectivity.
- **Friday: Real-World Architecture:** Architect a multi-datacenter replica set configuration for a banking system requiring zero data loss (`w:majority`) and sub-second disaster recovery.

Resources: MongoDB Replication docs (<https://www.mongodb.com/docs/manual/replication/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>).

Self-check:

- Why can `w:1` writes be rolled back during failover?
- What is the oplog, and why must updates be idempotent?
- Why are arbiters a risk in majority elections?
- What does `j:true` add to a write concern?
- Which read concern avoids reading rolled-back data?

Production Note: Write-concern trade-offs under failover: `w:1` writes acknowledged on the old primary can be rolled back when a new primary is elected; only `w:majority` (typically with `j:true`) guarantees durability across elections. Pair `w:majority` writes with `readConcern: "majority"` (and retryable writes) if you must never read rolled-back data, and budget the added latency.

Week 6: Sharding Architecture & Chunk Allocations

Outcomes: identify the roles of `mongos`, CSRS, and shards, compare hash vs. range shard keys, explain chunk splitting and balancer movement, diagnose a hot shard from a monotonic key.

- **Monday:** Sharding Components: `mongos` routers, Config Server Replica Sets (CSRS), and Shard nodes.
- **Tuesday:** Shard Keys: Range-based vs. Hash-based vs. Compound Shard Keys.
- **Wednesday:** Chunk Mechanics: Chunk sizes, Auto-splitting, and Balancer operations.
- **Thursday: Hands-on Project:** Deploy a 2-shard cluster with CSRS and `mongos`. Enable sharding on a collection, configure Hash-based sharding on `_id`, and monitor chunk distribution.
- **Friday: Real-World Architecture:** Diagnose a production sharding failure caused by a monotonically increasing shard key (`createdAt`) creating hot-shard write bottlenecks.

Resources: MongoDB Sharding docs (<https://www.mongodb.com/docs/manual/sharding/>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>). **Self-check:**

- What roles do `mongos` and the CSRS play?
- Hash-based vs. range-based shard keys: what is the trade-off?
- Why does a monotonically increasing shard key create a hot shard?
- What is a chunk, and when is it split?
- What does the balancer move, and at what cost?

Production Note: Chunk migrations are not free: each move copies data and briefly locks source chunks, adding read/write latency on the donor shard. Restrict the balancer to a low-traffic window (`sh.setBalancerState` / balancing window settings) during peak-sensitive periods, and alert on excessive migration rates as a symptom of a poor shard key.

Week 7: Cluster Balancing & Shard Key Optimization

Outcomes: score shard keys on cardinality, frequency, and monotonicity, refine a shard key live with `refineCollectionShardKey`, distinguish targeted from scatter-gather queries via `explain()`.

- **Monday:** Evaluating Shard Keys: Cardinality, Frequency, and Rate of Change.

- **Tuesday:** Refining Shard Keys: Live shard key modification (`refineCollectionShardKey`).
- **Wednesday:** Scatter-Gather Queries vs. Targeted Single-Shard Queries.
- **Thursday: Hands-on Project:** Simulate a hot-shard scenario using a low-cardinality shard key. Use `refineCollectionShardKey` to add a suffix field live without cluster downtime.
- **Friday: Real-World Architecture:** Re-architect a 50TB sharded telemetry cluster to eliminate scatter-gather queries for 90% of analytical read requests.

Resources: MongoDB shard key guidance (<https://www.mongodb.com/docs/manual/core/sharding-shard-key/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>).

Self-check:

- Name the three properties of a good shard key.
- What does `refineCollectionShardKey` allow without downtime?
- Why are scatter-gather queries expensive?
- Why is a low-cardinality shard key problematic?
- What metric pattern reveals a hot shard?

Week 8: Global Clusters & Multi-Region Architectures

Outcomes: configure zone sharding for data residency, pin EU chunks to EU shards and verify placement, choose read preferences and tag sets for locality, design a GDPR-compliant global cluster.

- **Monday:** Zone Sharding: Pinning specific data chunks to geographical shards.
- **Tuesday:** Global Clusters in MongoDB Atlas: Local reads/writes with global consistency.
- **Wednesday:** Read Preferences (`primaryPreferred`, `secondaryPreferred`, `nearest`) and Tag Sets.
- **Thursday: Hands-on Project:** Configure Zone Sharding on a Global User collection. Pin European users to `Shard_EU` and US users to `Shard_US` based on `country_code`.
- **Friday: Real-World Architecture:** Architect a GDPR-compliant global platform requiring European user data to reside strictly within EU physical borders while allowing global analytics queries.

Resources: MongoDB Global Clusters / zone sharding docs (<https://www.mongodb.com/docs/atlas/global-clusters/>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>).

Self-check:

- What does zone sharding pin to a shard?
- How do you enforce EU-only residency for EU user data?
- What does the `nearest` read preference do?
- What are tag sets used for?
- Why do global clusters favor local reads and writes?

Acceptance

- Cluster mongos, CSRS.
- Architecture CSRS.
- At least one check, CSRS.
- Monitoring export.
- Monthly update.
- Security handling n

CAPSTONE PROJECT - Milestone 2: Globally Distributed Multi-Region Sharded Cluster. Deploy a 4-shard MongoDB cluster spanning 3 simulated cloud regions. Implement Zone Sharding to partition user data by continent. Refine shard keys on a live database with 20 million documents, measure scatter-gather query reduction, and simulate an entire region outage while maintaining write availability.

Milestone 2 Capstone: Reference Solution Sketch

- **Architecture:** 4-shard Atlas cluster across 3 regions behind mongos, CSRS for metadata, zone sharding pinning user chunks by `country_code` — chosen because zone ranges give auditable residency guarantees.
- Provision shards and CSRS first; add zones and chunk ranges *before* loading data so documents never land outside their region.
- Load 20M synthetic users (`mgeneratejs`); verify distribution with `sh.status()` and per-zone counts.
- Refine the shard key live (`refineCollectionShardKey` adding a random suffix) to remove the hot shard; measure scatter-gather before/after.

```
// Residency verification: every EU chunk lives only on EU shards
sh.addShardToZone("shardEU", "EU");
sh.updateZoneKeyRange("app.users",
  { country_code: "DE" }, { country_code: "DF" }, "EU");
// then assert: sh.status() shows no EU-tagged chunk on a non-EU shard
```

- Simulate a region outage (stop one shard's nodes); confirm `w:majority` writes continue via remaining electors.
- **Pitfall:** assigning zones after bulk load triggers a massive chunk-migration storm — zone first, load second.
- **Pitfall:** a 2-node region cannot form a majority alone — place electors across at least 3 fault domains.
- **Pitfall:** measuring scatter-gather only by latency hides cost — count shards hit in `explain("executionS`

3 Milestone 3: Aggregation Framework, Indexes & Performance Tuning (Month 3)

Objective: Master compound index mechanics, analytical aggregation pipelines, and query execution profiling.

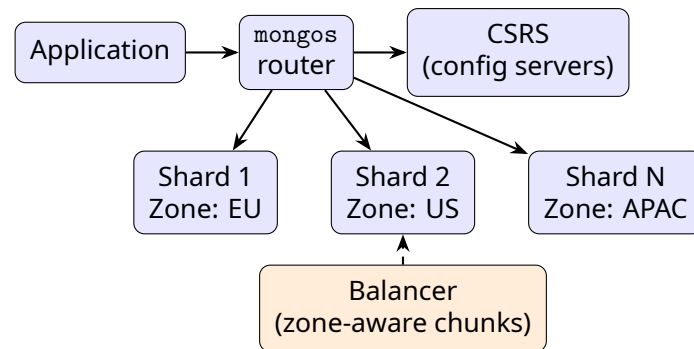


Figure 3: Milestone 2 reference architecture: sharded cluster with `mongos` routing, CSRS meta-data, and zone-labeled shards for data residency.

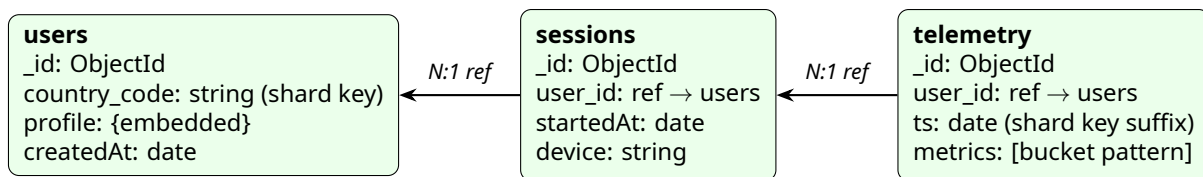


Figure 4: Capstone data model: Globally Distributed Sharded Cluster — shard-key fields marked; referencing keeps user documents small while zone sharding pins residency.

Learning outcomes: design ESR-compliant compound indexes, build multi-stage pipelines with `$lookup/$facet/$setWindowFields`, read `explain()` plans, eliminate slow queries and unused indexes.

Week 9: Index Mechanics & Query Optimizer

Outcomes: select single/compound/multikey/partial index types, apply the ESR rule to compound index design, prove coverage with `explain("executionStats")`, plan a rolling index build safely.

- **Monday:** Index Types: Single Field, Compound, Multikey (Arrays), Partial, Sparse, and TTL Indexes.
- **Tuesday:** The ESR Rule: Equality, Sort, Range order in Compound Index design.
- **Wednesday:** Covered Queries, Index Intersection, and Indexing Wildcards (`$**`).
- **Thursday: Hands-on Project:** Create a 5-million document collection. Write queries violating the ESR rule, analyze index usage via `explain("executionStats")`, and build an optimal compound index.
- **Friday: Real-World Architecture:** Optimize an enterprise search pipeline that is suffering from high CPU usage caused by un-indexed array queries (Multikey Index Bounds).

Resources: MongoDB indexing docs & ESR rule (<https://www.mongodb.com/docs/manual/tutorial/equality-sort-range-rule/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>). **Self-check:**

- State the ESR rule for compound index field order.
- What makes a query “covered”?
- What restriction applies to array fields in compound indexes?

- When is a partial index preferable to a full index?
- Why use rolling index builds on large replica sets?

Production Note: Building an index on a busy primary blocks or degrades writes and spikes WiredTiger cache pressure. On replica sets, prefer rolling index builds (build on each secondaries-first via standalone mode, then step down the primary) for large collections, and always schedule builds in a low-traffic window with `currentOp` monitoring.

Week 10: Aggregation Pipeline Mastery - Stage 1

Outcomes: compose core stages with early `$match`, reshape documents with `$unwind/$facet`, implement `$lookup` and `$graphLookup` joins, enable `allowDiskUse` deliberately.

- **Monday:** Core Stages: `$match`, `$project`, `$group`, `$sort`, and `$limit`.
- **Tuesday:** Complex Reshaping: `$unwind`, `$arrayToObject`, `$objectToArray`, and `$facet`.
- **Wednesday:** Joins in MongoDB: `$lookup` (Uncorrelated, Correlated, Subqueries) and `$graphLookup`.
- **Thursday: Hands-on Project:** Write a `$graphLookup` pipeline over an organizational hierarchy collection to compute deep reporting chains and total team headcount recursively.

```
pipeline = [
  {"$match": {"order_date": {"$gte": start, "$lt": end}}},
  {"$lookup": {
    "from": "customers",
    "localField": "customer_id",
    "foreignField": "_id",
    "as": "customer"}},
  {"$unwind": "$customer"},
  {"$facet": {
    "revenue_by_region": [
      {"$group": {"_id": "$customer.region",
        "revenue": {"$sum": "$total"},
        "orders": {"$count": {}}}},
      {"$sort": {"revenue": -1}}],
    "top_products": [
      {"$unwind": "$items"},
      {"$group": {"_id": "$items.sku",
        "units": {"$sum": "$items.qty"}}},
      {"$sort": {"units": -1}},
      {"$limit": 10}],
    "summary": [
      {"$group": {"_id": None,
        "total_revenue": {"$sum": "$total"}}}]
  ]
}
results = list(db.orders.aggregate(pipeline, allowDiskUse=True))
```

- **Friday: Real-World Architecture:** Design an analytical pipeline that replaces 10 disparate SQL JOIN queries with a single optimized `$lookup` and `$facet` pipeline.

Resources: MongoDB aggregation pipeline docs (<https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-h>)

Self-check:

- Why place `$match` as early as possible in a pipeline?
- What does `$facet` do in a single pass?

- When do you need `$graphLookup` instead of `$lookup`?
- What does `$unwind` do to an array field?
- What does `allowDiskUse` permit, and at what cost?

Week 11: Aggregation Pipeline Mastery - Stage 2

Outcomes: compute windowed metrics with `$setWindowFields`, choose accumulators per stage context, respect the 100MB stage RAM limit, build moving-average and ranking pipelines.

- **Monday:** Window Functions: `$setWindowFields`, `$denseRank`, `$derivative`, and `$exponentialMovingAverage`.
- **Tuesday:** Accumulators in `$group` vs. `$project` vs. `$setWindowFields`.
- **Wednesday:** Pipeline Performance: RAM limits (100MB stage limit), `allowDiskUse`, and pipeline optimization.
- **Thursday: Hands-on Project:** Build a financial analytics pipeline using `$setWindowFields` to calculate a 30-day moving average and exponential smoothing over transaction logs.
- **Friday: Real-World Architecture:** Architect a real-time leaderboard and metrics processing engine using `$setWindowFields` over a stream of 50 million gaming event documents.

Resources: MongoDB `$setWindowFields` docs (<https://www.mongodb.com/docs/manual/reference/operator/aggregation/setWindowFields/>); Apache Spark repo for batch-window comparisons (<https://github.com/apache/spark>). **Self-check:**

- What class of calculations does `$setWindowFields` enable?
- What happens when a stage exceeds the 100MB RAM limit?
- What is the difference between `$rank` and `$denseRank`?
- What does `$derivative` compute over a window?
- How is a 30-day moving-average window specified?

Production Note: Pipelines that exceed the 100MB per-stage RAM limit abort unless `allowDiskUse` is set — but disk-spilling sorts/groups are 10–100x slower. Push selective `$match` and `$project` stages as early as possible, and treat heavy disk spilling in slow logs as a capacity-planning red flag.

Week 12: Performance Profiling & Query Tuning

Outcomes: enable and query the database profiler, read IXSCAN vs. COLLSCAN plans, prune unused indexes with `$indexStats`, diagnose queued-ticket saturation.

- **Monday:** Database Profiler (`db.setProfilingLevel`), System Profile Collection (`system.profile`).
- **Tuesday:** Reading `explain()` plans: IXSCAN, COLLSCAN, SORT_KEY_GENERATOR, and FETCH.
- **Wednesday:** Identifying bottlenecks: WiredTiger tickets, Slow Logs, Index Stats (`$indexStats`).
- **Thursday: Hands-on Project:** Profile an un-optimized database. Identify unindexed queries, remove redundant/unused indexes using `$indexStats`, and fix memory spill warnings.
- **Friday: Real-World Architecture:** Perform an architectural audit on a MongoDB cluster experiencing high `qr/qw` (queued read/write) tickets, tuning slow queries and index structures.

Resources: MongoDB database profiler & explain() docs (<https://www.mongodb.com/docs/manual/tutorial/manage-the-database-profiler/>); DuckDB repo for analytical benchmarking comparisons (<https://github.com/duckdb/duckdb>). **Self-check:**

- What do IXSCAN vs. COLLSCAN in explain() mean?
- What does the system.profile collection store?
- What does \$indexStats help you find?
- What do high qr/qw ticket counts indicate?
- What does SORT_KEY_GENERATOR in a plan signal?

CAPSTONE PROJECT - Milestone 3: Real-Time Analytical Engine over 100 Million Documents. Build a multi-stage aggregation pipeline calculating rolling financial metrics, multi-faceted customer filters, and organizational hierarchy trees using \$graphLookup and \$setWindowFields. Optimize execution using ESR-compliant compound indexes, ensuring all queries are fully covered with zero disk spilling.

Acceptance Criteria

- Pipeline doc da
- Architecture design
- At least dinality
- Monitor plans a
- Monthly docum
- Security noted.

Milestone 3 Capstone: Reference Solution Sketch

- **Architecture:** a dedicated analytics replica set (or analytics nodes) serving aggregation pipelines over ESR compound indexes, with \$merge writing precomputed results — chosen to isolate analytical load from OLTP primaries.
- Generate 100M documents; profile the naive pipeline first to record a COLLSCAN baseline.
- Design one compound index per dominant query shape using the ESR rule; validate coverage via explain("executionStats").
- Build the pipeline: early \$match → \$lookup → \$facet → \$setWindowFields → \$merge.

```
# Hardest part: a fully covered rolling-metric window over 100M docs
pipeline = [
  {"$match": {"ts": {"$gte": start}}},          # index range
  {"$setWindowFields": {
    "partitionBy": "$account_id",
    "sortBy": {"ts": 1},
    "output": {"avg30d": {"$avg": "$amount",
      "window": {"range": [-30, 0], "unit": "day"}}}},
  {"$merge": {"into": "acct_metrics",
    "on": "_id", "whenMatched": "replace"}}]
```

- **Pitfall:** \$lookup before \$match joins 100M docs needlessly — filter first.

- **Pitfall:** `$setWindowFields` requires a sort; an unsorted window returns nondeterministic results.
- **Pitfall:** “covered” is lost the moment the projection includes an unindexed field — verify zero FETCH stages.

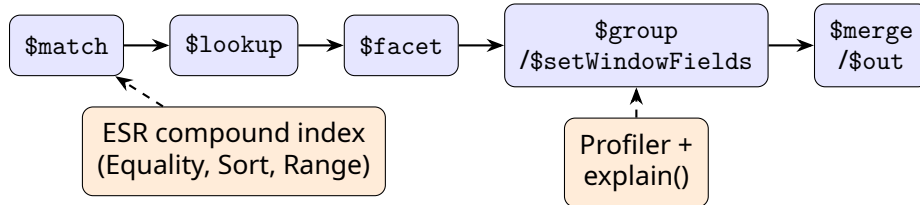


Figure 5: Milestone 3 reference architecture: aggregation pipeline stage flow with ESR index guidance and profiling feedback loop.

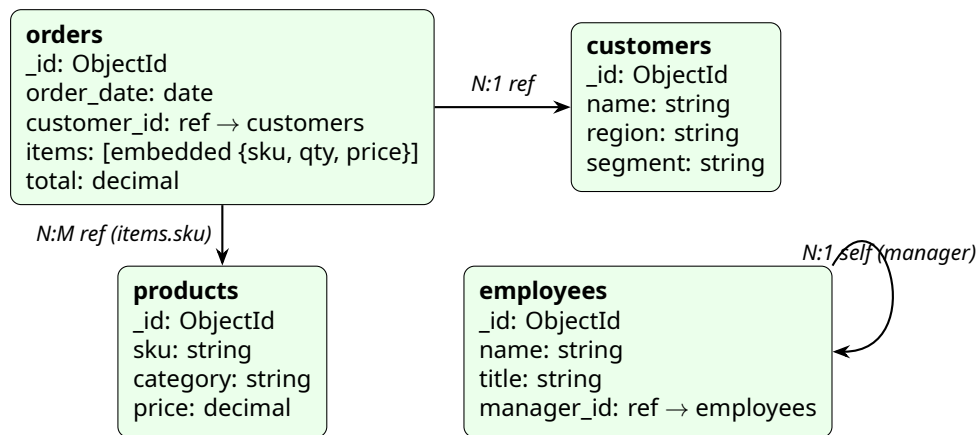


Figure 6: Capstone data model: Real-Time Analytical Engine — orders join customers and products via `$lookup`; the employees self-reference powers `$graphLookup` hierarchies.

4 Milestone 4: Atlas Ecosystem, Search, Vector Search & Streaming (Month 4)

Objective: Master Lucene-powered Atlas Search, Atlas Vector Search, Atlas Stream Processing, and Event Services.

Learning outcomes: build Atlas Search indexes with custom analyzers, implement Hybrid Search (BM25 + vectors), author windowed Atlas Stream Processing pipelines, consume Change Streams with resumability.

Week 13: Atlas Search (Lucene Integration)

Outcomes: define Atlas Search indexes with custom analyzers, compose compound queries with `autocomplete` and `phrase`, implement faceting and relevance boosting in `$search`.

- **Monday:** Atlas Search Architecture: Apache Lucene integration and `$search` pipeline stage.
- **Tuesday:** Index Definitions: Analyzers, Tokenizers (Standard, Keyword, Ngram), and Field Mappings.

- **Wednesday:** Operators: text, phrase, wildcard, autocomplete, and compound queries.
- **Thursday: Hands-on Project:** Create an Atlas Search Index with Custom Analyzers. Write a `$search` pipeline using `autocomplete` for a real-time product search box.
- **Friday: Real-World Architecture:** Architect an e-commerce search platform featuring fuzzy matching, relevance boosting, facets, and highlighting across a 10-million item catalog.

Resources: Atlas Search docs (<https://www.mongodb.com/docs/atlas/atlas-search/>); Apache Kafka repo (<https://github.com/apache/kafka>). **Self-check:**

- Which engine powers Atlas Search?
- Which aggregation stage invokes Atlas Search?
- Which operator suits type-ahead search boxes?
- What does an analyzer do to indexed text?
- When would you choose an ngram tokenizer?

Week 14: Atlas Vector Search & GenAI Architectures

Outcomes: explain embeddings and distance metrics, create HNSW vector indexes, run `$vectorSearch` queries, combine BM25 and vector results into a hybrid RAG retrieval stage.

- **Monday:** Vector Search Foundations: Embeddings, High-dimensional spaces, and Distance Metrics (Cosine, Dot Product, Euclidean).
- **Tuesday:** Vector Indexing: Hierarchical Navigable Small World (HNSW) and Inverted File (IVF) indexes.
- **Wednesday:** The `$vectorSearch` Stage: Hybrid Search combining `$search` (BM25) and `$vectorSearch`.
- **Thursday: Hands-on Project:** Build a Retrieval-Augmented Generation (RAG) backend. Store HuggingFace text embeddings in MongoDB Atlas, build a Vector Index, and query using `$vectorSearch`.
- **Friday: Real-World Architecture:** Architect an enterprise semantic search and recommendation engine performing Hybrid Search over unstructured PDF documents and structural metadata.

Resources: Atlas Vector Search docs (<https://www.mongodb.com/docs/atlas/atlas-vector-search/vector-search-overview/>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer>) **Self-check:**

- What is an embedding?
- What property makes HNSW indexes fast?
- What does hybrid search combine?
- When is cosine distance preferred over Euclidean?
- What role does `$vectorSearch` play in a RAG pipeline?

Week 15: Atlas Stream Processing (ASP) & Event-Driven Pipelines

Outcomes: stand up an ASP workspace with Kafka/Change-Stream connections, implement tumbling/hopping window aggregations, manage window/checkpoint state growth, join streams against Atlas collections with `$lookup`, build end-to-end exactly-once sinks via checkpointing + idempotent `$merge` upserts on a dedup key, route poison events to a dead-letter queue.

- **Monday:** Atlas Stream Processing (ASP) Concepts: Streams, Workspaces, Connections, and Pipelines.
- **Tuesday:** Streaming Ingestion: Consuming from Kafka, Kinesis, and MongoDB Change Streams. **Stream joins:** `$lookup` against Atlas collections inside ASP pipelines (stream-collection enrichment), time-windowed stream-stream joins, and late-arrival implications for join matches.
- **Wednesday:** Windowing in ASP: Tumbling, Hop, and Session Windows. **State management:** how ASP maintains window and checkpoint state, checkpoint intervals and their latency/recovery trade-off, state growth from high-cardinality group keys, and why unbounded state kills streaming jobs (memory pressure, slow checkpoints, restart storms).
- **Thursday: Hands-on Project:** Set up an Atlas Stream Processing pipeline consuming from a Kafka topic, aggregating events over 5-minute Tumbling Windows, and merging into an Atlas collection. Make it end-to-end exactly-once: checkpointing plus an idempotent `$merge` upsert keyed on a deterministic dedup key, with a dead-letter queue for poison events.

```
// ASP pipeline: enrich events, then exactly-once sink via idempotent upsert
pipeline = [
  { $source: { connectionName: "kafka_orders" } },
  { $lookup: {
    // stream-collection join
    from: { connectionName: "atlas_db", db: "shop", coll: "customers" },
    localField: "customer_id", foreignField: "_id", as: "customer" } },
  { $set: { dedup_key: { $concat: ["$customer_id", ":", "$event_id"] } } },
  { $merge: {
    // exactly-once sink
    into: { connectionName: "atlas_db", db: "shop", coll: "enriched_orders" },
    on: "dedup_key",
    // upsert on the dedup key
    whenMatched: "replace", whenNotMatched: "insert" } }
]
```

- **Friday: Real-World Architecture:** Design a real-time fraud detection pipeline processing streaming credit card events via ASP — enriching each event with cardholder data via `$lookup`, joining against a velocity stream over a time window — before persisting materialized summary alerts to MongoDB with idempotent upserts and a dead-letter queue for malformed events.

Resources: Atlas Stream Processing docs (<https://www.mongodb.com/docs/atlas/atlas-stream-processing>)
Apache Kafka repo (<https://github.com/apache/kafka>). **Self-check:**

- Why does unbounded state from high-cardinality keys kill a streaming job?
- What does `$lookup` enable inside an ASP pipeline?
- What do late-arriving events imply for time-windowed joins?
- How do checkpointing + idempotent `$merge` deliver end-to-end exactly-once?
- What happens when a resume token outlives the oplog window?

Production Note: Streaming sinks must be idempotent: at-least-once delivery from Kafka/ASP means duplicate window emissions. Use deterministic `_id` keys (window start + group key) with `$merge upserts`, route poison events to a dead-letter topic, and alert on consumer lag rather than silently falling behind.

Production
retention w
re-bootstrap
dump of t
resume tok
case down

Week 16: Atlas Data Services, Serverless Integration & Reverse ETL

Outcomes: build resumable Change Stream consumers with persisted tokens, trigger Atlas Functions on writes, implement a reverse-ETL sync to a CRM with identity resolution, size serverless connection pools.

- **Monday:** MongoDB Change Streams: Resume tokens, pre/post-images, and event filters.
- **Tuesday:** Atlas Triggers & Functions: Event-driven serverless functions on DB writes. **Reverse ETL:** pushing curated aggregates (e.g. customer-360 scores) back into operational systems via Change Streams + Atlas Functions calling a CRM API — identity resolution (matching on stable keys / hashed emails) and sync-frequency trade-offs (real-time vs. batch).
- **Wednesday:** Atlas Data API & GraphQL API: HTTPS-based database interactions.
- **Thursday: Hands-on Project:** Build a microservice utilizing Change Streams to listen for document updates, triggering an Atlas Function to sync changes to an external search index.

```
from pymongo import MongoClient

client = MongoClient("mongodb+srv://<cluster>.mongodb.net/")
collection = client.shop.orders
resume_token = load_resume_token() # persisted checkpoint, e.g. in a collection

pipeline = [{"$match": {"operationType": {"$in": ["insert", "update"]}}}]
try:
    with collection.watch(pipeline, resume_after=resume_token) as stream:
        for change in stream:
            doc = change["fullDocument"]
            sync_to_search_index(doc) # external index update
            save_resume_token(stream.resume_token) # checkpoint AFTER processing
except Exception:
    # Re-open with the last saved resume token; no events are lost.
    with collection.watch(pipeline, resume_after=load_resume_token()) as stream:
        for change in stream:
            sync_to_search_index(change["fullDocument"])
            save_resume_token(stream.resume_token)
```

- **Friday: Real-World Architecture:** Architect an event-driven serverless platform utilizing Change Streams, Atlas Triggers, and ASP to process order fulfillment events asynchronously.

Resources: MongoDB Change Streams docs (<https://www.mongodb.com/docs/manual/changeStreams/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>).

Self-check:

- What does a change-stream resume token identify?

- Why persist the resume token only after processing an event?
- What is reverse ETL?
- What problem does identity resolution solve in CRM syncs?
- Real-time vs. batch sync: what is the trade-off?

Production Note: Connection pool sizing for serverless functions: each cold-starting function opens a fresh connection pool, and a burst of hundreds of concurrent functions can exhaust `maxPoolSize × instances` against the cluster's connection limit. Reuse clients across warm invocations (global scope), set a small `maxPoolSize` (e.g. 1–10) for short-lived functions, and prefer the Atlas Data API or connection pooling proxies for spiky workloads.

Atlas option	Billing model	Best for	Watch-outs
Atlas Flex	Low fixed monthly fee	Dev/test, small apps, early milestones of this course	5GB storage cap, limited metrics, backup/peering options
Atlas Serverless	Per-operation (read/write units) + storage	Spiky, unpredictable, or idle-heavy workloads	Cold-start latency, custom WiredTiger or sharding knobs
Dedicated M10	Fixed hourly per node	Baseline production: full metrics, backups, PITR	Paying for idle capacity off-peak; RAM/IOPS headroom
Dedicated M30+	Higher fixed hourly per node	Sharded/heavy workloads, NVMe storage, large caches	Cost scales fast by region; right-sizing, scaling out

CAPSTONE PROJECT - Milestone 4: Agentic E-Commerce Recommendation & Search Engine. Build an advanced Atlas backend. Ingest product catalogs into MongoDB, index embeddings with Atlas Vector Search, implement Hybrid Search (`$search + $vectorSearch`), and process streaming user activity using Atlas Stream Processing over 5-minute sliding windows to generate real-time personalized recommendations.

Acceptance Criteria

- End-to-end pipeline: ingestion + indexing
- Architecture: ASP data plane, Atlas Search
- At least one recommendation variant
- Monitor recommendations: shot on target
- Monthly recommendations: decision quality
- Security: user activity, recommendations

Milestone 4 Capstone: Reference Solution Sketch

- **Architecture:** Atlas cluster → Change Streams + ASP (Kafka source) → windowed `$merge` into recommendations, served by a hybrid `$search + $vectorSearch` endpoint — one platform

form for OLTP, search, and streaming.

- Seed the catalog; generate 768-d embeddings per product and store `model_version` for re-embedding rollouts.
- Create the vector index (HNSW, cosine) and the search index (custom analyzer) via API scripts, not the UI.
- Implement hybrid retrieval: run `$vectorSearch` and `$search`, then merge with reciprocal-rank fusion.

```
# Reciprocal-rank fusion of BM25 and vector hits (the hardest part)
def hybrid(bm25, vec, k: int = 60) -> list:
    score = {}
    for rank, doc in enumerate(bm25):
        score[doc["_id"]] = score.get(doc["_id"], 0) + 1 / (k + rank + 1)
    for rank, doc in enumerate(vec):
        score[doc["_id"]] = score.get(doc["_id"], 0) + 1 / (k + rank + 1)
    return sorted(score, key=score.get, reverse=True)[:20]
```

- ASP: consume clickstream events, aggregate per user in 5-minute windows, `$merge` upserts keyed on `user_id + window_start`.
- **Pitfall:** embedding dimension drift (768 vs. 1024) silently zeroes recall — validate dimensions in CI.
- **Pitfall:** windowed `$merge` without deterministic `_id` duplicates rows on replay.
- **Pitfall:** Atlas Search index rebuilds lag large bulk loads — poll index status before serving traffic.

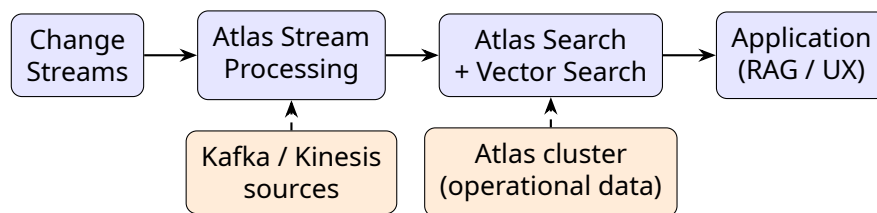


Figure 7: Milestone 4 reference architecture: Atlas ecosystem flow from Change Streams through ASP into Search/Vector Search serving the application.

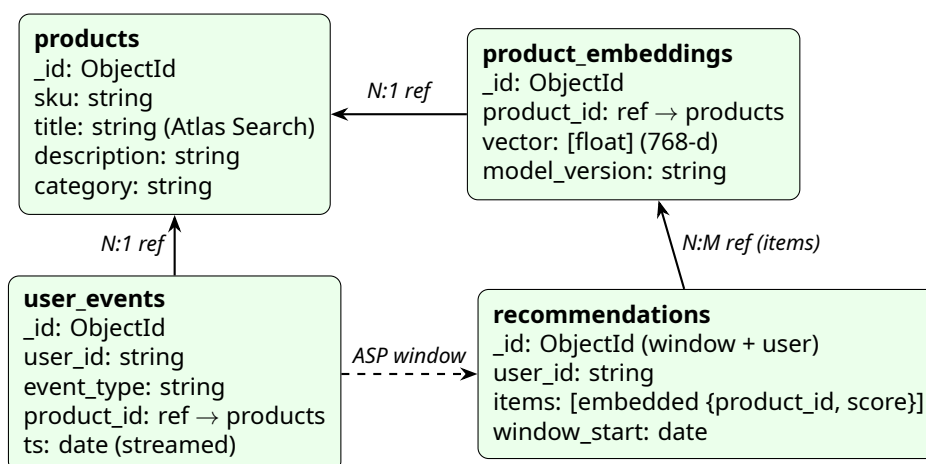


Figure 8: Capstone data model: Agentic E-Commerce Engine — embeddings reference products; streamed user events feed windowed recommendation documents.

5 Milestone 5: Enterprise Security, Governance, Encryption & Compliance (Month 5)

Objective: Master RBAC, Client-Side Field Level Encryption (CSFLE), Queryable Encryption (QE), and Auditing.

Learning outcomes: configure enterprise authentication and least-privilege RBAC, implement CSFLE/QE with a KMS, enforce network isolation with PrivateLink, design audit and data-lifecycle governance.

Week 17: Authentication, Authorization & RBAC

Outcomes: configure SCRAM/x.509/LDAP authentication, design least-privilege custom roles per collection and action, restrict service accounts to specific pipeline stages, detect authorization-failure anomalies.

- **Monday:** Authentication Mechanisms: SCRAM-SHA-256, x.509 Certificates, LDAP, Kerberos, and OIDC.
- **Tuesday:** Role-Based Access Control (RBAC): Built-in roles, Custom Roles, and Privilege Actions.
- **Wednesday:** Database-level Privileges: Restricting access to specific collections and actions.
- **Thursday: Hands-on Project:** Deploy an x.509-authenticated MongoDB cluster. Create custom RBAC roles using `grantPrivilegesToRole` to restrict a service user to specific pipeline stages.
- **Friday: Real-World Architecture:** Architect an enterprise security model integrating Active Directory (LDAP) with MongoDB Atlas, enforcing least-privilege role hierarchies.

Resources: MongoDB security checklist & RBAC docs (<https://www.mongodb.com/docs/manual/administration/security-checklist/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>). **Self-check:**

- SCRAM-SHA-256 vs. x.509: which suits services at scale?
- Why create custom RBAC roles?
- What does `grantPrivilegesToRole` modify?
- What is the least-privilege rule for service accounts?
- Why alert on spikes of authorization failures?

Production Note: *Least-privilege in practice: grant roles per collection and action, never `readWriteAnyDatabase` for service accounts. Rotate x.509 certificates and LDAP-bound credentials on a schedule, and alert on authorization failures — a spike usually means a misconfigured role or a probing attacker.*

Week 18: Client-Side Field Level Encryption (CSFLE) & Queryable Encryption

Outcomes: manage DEKs in a key vault wrapped by a KMS master key, choose deterministic vs. random encryption, run equality queries over QE-encrypted fields, execute GDPR erasure via crypto-shredding.

- **Monday:** CSFLE Architecture: Key Vault Collections, Data Encryption Keys (DEK), and Master Keys (KMS).
- **Tuesday:** Encryption Algorithms: Deterministic Encryption vs. Random Encryption.
- **Wednesday:** Queryable Encryption (QE): Searching encrypted fields directly using fast equality queries without decrypting on the server.
- **Thursday: Hands-on Project:** Configure Queryable Encryption (QE) in Python using AWS KMS. Encrypt sensitive PII fields (SSN, Credit Card), and run equality queries on the encrypted data.
- **Friday:** GDPR Right-to-Erasure via crypto-shredding: assign one DEK per data subject (`keyAltNames` = subject ID) in a KMIP/KMS provider, so destroying that single key renders every ciphertext unreadable — in the primary collection, across replica-set secondaries and sharded clusters, in Change Stream events consumed by downstream pipelines, in Atlas Search indexes (which embed the encrypted values), and inside continuous-backup snapshots and Atlas Online Archive tiers, without waiting for snapshot retention expiry. Trace the alternative — row-level `deleteMany` plus downstream replay — and its propagation burden through consumers, derived collections, and search re-indexing. Handle legal hold: keys under hold are quarantined (flagged in the KMS, never destroyed) until the hold is released, with every destroy/quarantine action written to the audit log for the compliance trail. **Real-World Architecture:** Design a HIPAA/PCI-DSS compliant database architecture for a healthcare provider, securing all patient records using Queryable Encryption with crypto-shredding as the erasure mechanism.

Resources: MongoDB CSFLE & Queryable Encryption docs (<https://www.mongodb.com/docs/manual/core/queryable-encryption/>); Data Engineer Handbook (<https://github.com/DataExpert-io/data-engineer-handbook>). **Self-check:**

- CSFLE vs. Queryable Encryption: what is the key difference?
- Deterministic vs. random encryption: what is the query implication?
- What is the relationship between a DEK and the KMS master key?
- Which query shapes does QE currently support?
- Why is losing the KMS master key catastrophic?

Production Note: Queryable Encryption adds storage overhead (encrypted indexes and meta-data collections can grow collections 2-4x) and restricts query shapes on encrypted fields. Encrypt only fields that genuinely require it, test query patterns before enabling QE in production, and protect KMS master keys with strict IAM — loss of the master key means unrecoverable data.

Production
continuous
emitted Ch
subject cry
row-level c
retention e
audit log.

Week 19: Network Security, PrivateLink & Auditing

Outcomes: isolate Atlas traffic with PrivateLink and security groups, enforce TLS 1.3 with custom CAs, write scoped JSON audit filters, ship audit logs to a SIEM with rotation.

- **Monday:** Network Isolation: IP Access Lists, VPC Peering, and AWS/Azure PrivateLink.
- **Tuesday:** TLS/SSL Configuration: Enforcing TLS 1.3 and custom Certificate Authorities.

- **Wednesday:** MongoDB Auditing Framework: Audit filters, capturing DDL/DML actions, and log rotation.
- **Thursday: Hands-on Project:** Enable auditing on a MongoDB Enterprise deployment. Write a custom JSON audit filter to capture all read/write attempts on a sensitive `salaries` collection.
- **Friday: Real-World Architecture:** Architect a zero-trust network perimeter for MongoDB Atlas using AWS PrivateLink, strict security groups, and real-time SIEM audit integration.

Resources: Atlas PrivateLink & network security docs (<https://www.mongodb.com/docs/atlas/security-private-endpoint/>); MongoDB auditing docs (<https://www.mongodb.com/docs/manual/core/auditing/>). **Self-check:**

- Why is PrivateLink preferred over IP access lists?
- Why enforce TLS 1.3 with custom certificate authorities?
- What do audit filters limit?
- Why ship audit logs off-host to a SIEM?
- Name the layers of a zero-trust Atlas perimeter.

Production Note: *Audit logging at high verbosity generates significant I/O and log volume. Use tight audit filters (sensitive collections and DDL actions only), ship audit logs off-host to a SIEM with rotation, and treat audit gaps (agent restarts, disk-full events) as incidents with a written runbook.*

Week 20: Governance, Archiving & Lifecycle Management

Outcomes: configure Atlas Online Archive with partition-aligned date keys, run federated queries across live and archived data, explain PITR mechanics and retention, verify restores on a schedule.

- **Monday:** Atlas Online Archive: Cold data tiering to S3/Azure Blob as Parquet.
- **Tuesday:** Federated Database Instances: Querying live MongoDB collections alongside S3 Parquet archives seamlessly.
- **Wednesday:** Backup Strategies: Cloud Backups, Point-In-Time Recovery (PITR), and snapshot management.
- **Thursday: Hands-on Project:** Configure Atlas Online Archive on a time-series collection. Write a Federated Query seamlessly reading active Atlas data and archived S3 Parquet data.
- **Friday: Real-World Architecture:** Design an enterprise data retention lifecycle automatically archiving 7-year-old financial logs to Cloud Storage while preserving queryability.

Resources: Atlas Online Archive docs (<https://www.mongodb.com/docs/atlas/online-archive/manage-online-archive/>); Apache Iceberg repo for open-table-format comparison (<https://github.com/apache/iceberg>). **Self-check:**

- What does Atlas Online Archive do?
- Why partition archives on the query date key?
- What can an Atlas federated query read together?

- What does PITR retention define?
- Why must restores be tested regularly?

Production Note: *Cost guardrails: Online Archive queries against cold Parquet are billed per GB scanned — always partition archives on the date/partition key used by queries. Verify restores regularly: a backup or archive that has never been restored is not a recovery strategy.*

CAPSTONE PROJECT - Milestone 5: Hardened Enterprise Healthcare Platform. Build a zero-trust MongoDB security architecture. Implement SCRAM + x.509 authentication, deploy Queryable Encryption (QE) using AWS KMS for patient PII fields, lock down network boundaries using PrivateLink, configure custom audit logging for compliance, and set up Atlas Online Archive to tier old records to S3.

Acceptance

- Platform
- Architecture work to
- At least trip qu
- Monitoring config)
- Monthly decision
- Security dling d

Milestone 5 Capstone: Reference Solution Sketch

- **Architecture:** PrivateLink-isolated Atlas cluster → x.509/SCRAM auth → least-privilege RBAC → Queryable Encryption (AWS KMS) on PII fields → audit trail to SIEM, with Online Archive for 7-year retention — defense in depth, each layer independent.
- Create the KMS master key with a deny-all-except-service IAM policy; generate one DEK per encrypted field with keyAltNames.
- Enable QE on `patients.name` and `patients.ssn`; round-trip equality queries through the encrypted client.
- Configure the audit filter to capture DDL plus all access to sensitive collections; ship logs to the SIEM.

```
# Per-subject crypto-shredding: one DEK per patient; destroy to erase
dek_id = client_encryption.create_data_key(
    kms_provider="aws", master_key=MASTER_KEY,
    key_alt_names=[f"patient:{patient_id}"]) # GDPR-ready

def erase_subject(patient_id: str) -> None:
    kv = db.get_collection("key_vault")
    dek = kv.find_one({"keyAltNames": f"patient:{patient_id}"})
    kv.delete_one({"_id": dek["_id"]}) # all copies, incl. backups, unreadable
    audit.log("crypto_shred", subject=patient_id)
```

- **Pitfall:** QE metadata collections triple storage — budget IOPS and test query shapes before enabling.
- **Pitfall:** shared DEKs across subjects make per-subject erasure impossible — key per subject from day one.

- **Pitfall:** audit filters that are too broad drown the SIEM and fill disks — scope to sensitive collections only.

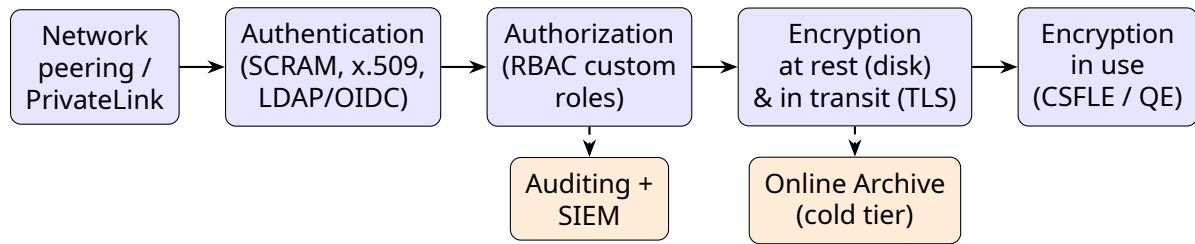


Figure 9: Milestone 5 reference architecture: layered security from network isolation through authentication, authorization, and encryption at rest, in transit, and in use.

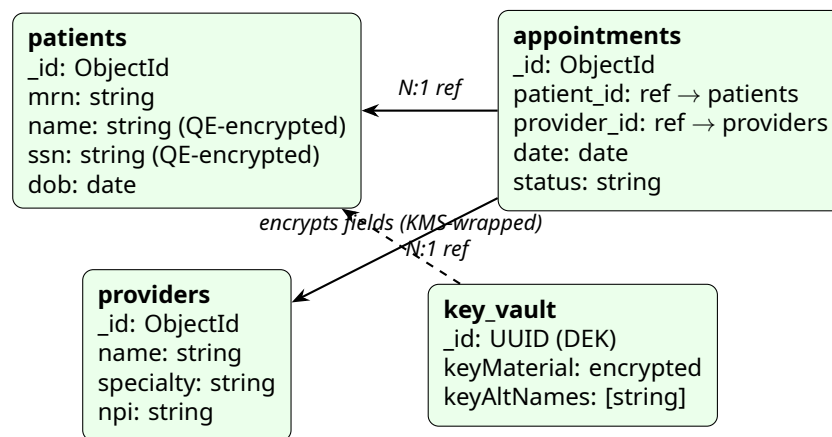


Figure 10: Capstone data model: Hardened Healthcare Platform — QE-encrypted PII fields in patients, referenced by appointments; DEKs held in the key vault collection.

6 Milestone 6: DataOps, Automation, Kubernetes & Certification (Month 6)

Objective: Master Infrastructure as Code, Kubernetes Operator, Ops Manager, and DBA/Data Engineer Certification.

Learning outcomes: provision Atlas infrastructure with Terraform, operate MongoDB on Kubernetes via the Operator, execute PITR restore drills, pass the certification practice exam.

Week 21: Infrastructure as Code (Terraform & CloudFormation)

Outcomes: provision Atlas projects, clusters, users, and private endpoints with Terraform, automate index and \$jsonSchema migrations in CI/CD, execute zero-downtime expand-contract schema evolution.

- **Monday:** MongoDB Atlas Terraform Provider: Provisioning Clusters, Projects, and Users.
- **Tuesday:** Managing Database Access, Private Endpoints, and Search Indexes via Terraform.
- **Wednesday:** Automating Index Deployments using `mongosh` scripts in CI/CD pipelines.
- **Thursday: Hands-on Project:** Write Terraform HCL scripts to provision an Atlas Sharded Cluster, configure VPC Peering, deploy database users, and automate index deployment.

```
resource "mongodbatlas_cluster" "analytics" {
  project_id = var.atlas_project_id
  name       = "analytics-sharded"
  cluster_type = "SHARDED"

  replication_specs {
    num_shards = 2
    regions_config {
      region_name      = "US_EAST_1"
      electable_nodes = 3
      priority          = 7
      read_only_nodes = 0
    }
  }
}

provider_name      = "AWS"
provider_instance_size_name = "M30"
mongo_db_major_version = "8.0"
backup_enabled      = true
pit_enabled          = true

labels {
  key   = "environment"
  value = "production"
}
```

- **Friday: Zero-Downtime Schema Evolution (expand-migrate-contract):** **expand** — deploy an application version writing the new optional field alongside the old one; **migrate** — dual-write new writes and backfill existing documents with a batched aggregation pipeline (`$set + $merge`, throttled, idempotent, resumable); **switch reads** — flip consumers to the new field behind a feature flag, with a backward-compatible view shim projecting both shapes for lagging consumers; **contract** — only after zero lagging readers remain, enforce the field via `$jsonSchema` (validationAction graduating warn → error), then `$unset` the old field in a final batched pass. Sequence every step as a reviewed migration in the

same CI/CD pipeline that deploys indexes (Wednesday), never as ad-hoc `mongosh` against production. **Real-World Architecture:** Architect a multi-environment (Dev, Staging, Prod) GitOps infrastructure deployment pipeline managing MongoDB Atlas infrastructure using Terraform, with expand-migrate-contract migrations gated per environment.

Resources: MongoDB Atlas Terraform provider registry (<https://registry.terraform.io/providers/mongodb/mongodbatlas/latest/docs>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>). **Self-check:**

- What resources does the Atlas Terraform provider manage?
- Why store Terraform state remotely with locking?
- Why run `terraform plan` in CI on every PR?
- Where should Atlas API keys live?
- Why bundle the Week 3 contract test into infra pipelines?

Production Note: *IaC guardrails: store Terraform state remotely with locking, run `terraform plan` in CI on every PR, and never store Atlas API keys in code — use CI secrets. Add the schema-drift contract test from Week 3 to the same pipeline so index, schema, and infrastructure changes deploy as one reviewed unit.*

Production
the backfil
serving tra
new field a
the dashbo
contract p

Week 22: MongoDB Kubernetes Operator (Enterprise/Community)

Outcomes: explain CRDs and operator reconciliation loops, deploy replica sets and sharded clusters as custom resources, execute rolling upgrades and TLS rotation on Kubernetes, contrast the Operator with Ops Manager.

- **Monday:** MongoDB Kubernetes Operator Architecture: Custom Resource Definitions (CRDs).
- **Tuesday:** Deploying Replica Sets and Sharded Clusters on Kubernetes via Custom Resources.
- **Wednesday:** Automating Rolling Upgrades, TLS certificate rotation, and storage scaling on K8s.
- **Thursday: Hands-on Project:** Deploy the MongoDB Community Kubernetes Operator on a local Minikube cluster. Provision a 3-node Replica Set, trigger a rolling version upgrade, and monitor recovery.
- **Friday: Real-World Architecture:** Design a private cloud database-as-a-service (DBaaS) architecture leveraging the MongoDB Enterprise Kubernetes Operator and Ops Manager.

Resources: MongoDB Kubernetes Operator docs (<https://www.mongodb.com/docs/kubernetes-operator/>)
Apache Airflow repo for complementary orchestration (<https://github.com/apache/airflow>).

Self-check:

- What is a Kubernetes CRD?
- What does the MongoDB Kubernetes Operator automate?
- How does the operator perform a rolling upgrade?
- Why use Minikube for learning the operator?
- Operator vs. Ops Manager: who does what?

Week 23: Backup, Disaster Recovery & Ops Manager / Cloud Manager

Outcomes: describe Ops Manager/Cloud Manager agent roles, execute a PITR restore drill against a measured RPO/RTO, configure Atlas alerts for lag, connections, and backup failures, write runbook entries per alert.

- **Monday:** Ops Manager / Cloud Manager Architecture: Monitoring, Backup, and Automation Agents.
- **Tuesday:** Continuous Backups and Point-In-Time Recovery (PITR) mechanics.
- **Wednesday:** Atlas Observability: metrics dashboards, alert configurations (e.g., replication lag > 10s, connection saturation, disk usage), and the Performance Advisor / Index Ranking engines.
- **Thursday: Hands-on Project:** Backup/PITR restore drill: enable Atlas Continuous Backup on a test cluster, snapshot document counts, then *accidentally* drop a collection. Perform a point-in-time restore to 1 minute before the drop and verify restored document counts match the pre-drop snapshot. Then configure Atlas alerts: replication lag > 10s and connections > 80% of maximum.
- **Friday: Real-World Architecture:** Architect a disaster recovery strategy with an RPO of < 1 minute and RTO of < 15 minutes for a multi-region enterprise database cluster.

Resources: Atlas monitoring & alerts docs (<https://www.mongodb.com/docs/atlas/monitoring-alerts/>); Atlas backup & PITR docs (<https://www.mongodb.com/docs/atlas/backup-restore-cluster/>).

Self-check:

- Define RPO and RTO.
- How does PITR reconstruct any point in time?
- Which two alert thresholds were configured this week?
- What does the Atlas Performance Advisor do?
- Why alert on backup job failures themselves?

Production Note: Day-2 operations: every alert needs a runbook entry (owner, triage steps, escalation). Test the PITR drill quarterly — measure actual RTO against the stated objective — and alert on backup job failures themselves, since a silently broken backup is the worst kind of surprise.

Week 24: MongoDB Certified DBA & Data Engineer Readiness

Outcomes: map exam domains to the six milestones, score 75%+ on a timed 60-question practice exam, triage incorrect answers to source domains, assemble the capstone portfolio for interviews.

- **Monday:** Exam Review: Storage Engine, Memory, and System Performance Domain (25%).
- **Tuesday:** Exam Review: Replication, Consensus, and Sharding Mechanics Domain (30%).
- **Wednesday:** Exam Review: Aggregation Framework, Indexes, and Search Domain (25%).
- **Thursday: Hands-on Project:** Take a full-length 60-question MongoDB Certified Data Engineer practice exam under timed conditions. Review incorrect responses and analyze edge cases.

- **Friday:** Final Review and Certification Readiness Verification.

Resources: MongoDB University certification catalog (<https://learn.mongodb.com/>); Data Engineering Zoomcamp (<https://github.com/DataTalksClub/data-engineering-zoomcamp>). **Self-check:**

- Which exam domain carries the most weight?
- What practice-exam score signals certification readiness?
- What fraction of the readiness score comes from capstones?
- What is a sound time-boxing strategy for 60 questions?
- What should you do with every incorrect practice answer?

CAPSTONE PROJECT - Milestone 6: Production-Grade GitOps Data Platform Deployment. Build an automated database platform. Use Terraform to deploy an Atlas cluster, integrate the MongoDB Kubernetes Operator for local staging, deploy CI/CD pipelines for Index and Schema migrations, configure Continuous Backups with PITR, and complete a full-length DBA/Data Engineer practice evaluation.

Acceptance Criteria

- Full platform deployed to K8s manifest
- Architecture diagram submitted to
- At least one connectivity test
- Monitoring and alerting configured
- Monthly backup and restore test
- Security audit for PII handling

Milestone 6 Capstone: Reference Solution Sketch

- **Architecture:** Git repo (HCL + K8s manifests) → CI/CD → Terraform Atlas provider for prod, MongoDB K8s Operator for staging — one reviewed pipeline ships infrastructure, indexes, and schemas together.
- Structure the repo as `infra/` (Terraform, remote state + locking), `migrations/` (numbered index/schema scripts), and `k8s/` (operator manifests).
- CI stages: `terraform plan` on PR → contract tests → apply to staging (K8s operator) → smoke tests → manual gate → prod apply.
- Enable Continuous Backup + PITR on prod; add a scheduled restore-verification job that restores to a scratch cluster nightly.

```
# migrations/0007_expand_contract.js -- runs in CI, idempotent
db.orders.updateMany(
  { order_total: { $exists: false }, total_price: { $exists: true } },
  [ { $set: { order_total: "$total_price" } } ] ); // backfill
// contract phase (separate later migration, after readers switch):
db.runCommand({ collMod: "orders", validator: { $jsonSchema: {
  required: ["order_total"] } }, validationAction: "error" });
db.orders.updateMany({}, [ { $unset: "total_price" } ] );
```

- **Pitfall:** applying Terraform to prod before staging passes hides provider bugs — gate prod on staging smoke tests.
- **Pitfall:** a one-shot full-table backfill saturates the primary — batch, throttle, and make migrations resumable.
- **Pitfall:** a restore-verification job that never runs means PITR is a hope, not a control — alert on its failures.

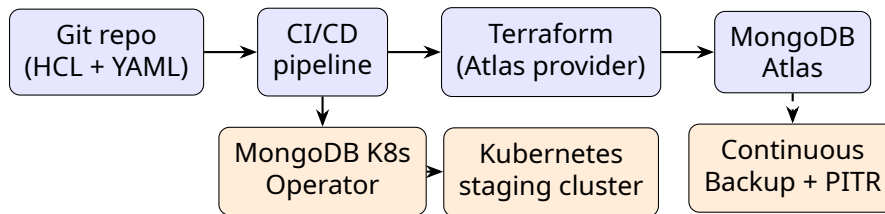


Figure 11: Milestone 6 reference architecture: GitOps flow from Git through CI/CD and Terraform into Atlas, with the Kubernetes Operator managing staging and Continuous Backup protecting production.

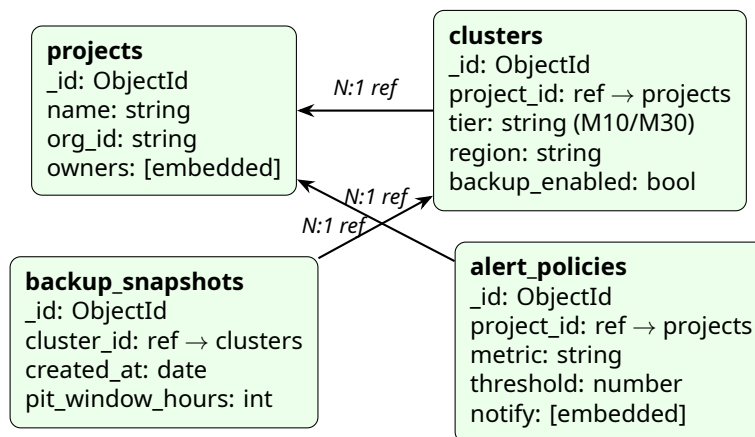


Figure 12: Capstone data model: Production GitOps Platform — Terraform-managed projects and clusters, with snapshot and alert-policy metadata tracked as documents.

7 Appendix A: Glossary

WiredTiger

MongoDB's default storage engine providing document-level concurrency control, compression, journaling, and checkpoints.

Checkpoint

A durable, consistent snapshot WiredTiger writes to disk every 60 seconds; journals cover the gaps between checkpoints.

Oplog

The capped operations log on replica set primaries that secondaries tail to replicate writes idempotently.

Replica Set

A group of mongod processes maintaining the same data set, providing automatic failover via Raft-like elections.

Write Concern

The acknowledgment level requested for a write (`w:1`, `w:majority`, `j:true`), trading latency for durability.

Read Concern

The consistency level of reads (`local`, `majority`, `linearizable`).

Read Preference

Routing rule for read traffic (e.g. `nearest`, `secondaryPreferred`) across replica set members.

Shard Key

The immutable-ish field(s) determining how documents distribute across shards; governs query targeting vs. scatter-gather.

Chunk

A contiguous range of shard-key values; the unit of data movement between shards during balancing.

Balancer

The background process that migrates chunks to keep shards evenly loaded; can be restricted to scheduled windows.

CSRS

Config Server Replica Set; stores cluster metadata (chunk ranges, zones) consulted by mongos.

mongos

The stateless query router for sharded clusters that targets queries using CSRS metadata.

Zone Sharding

Pinning chunk ranges to specific shards to enforce data residency or hardware affinity.

ESR Rule

Compound index field ordering guideline: Equality fields first, then Sort, then Range.

Covered Query

A query served entirely from an index without fetching documents.

Aggregation Pipeline

An ordered sequence of stages (`$match`, `$group`, `$lookup`, ...) transforming documents server-side.

\$facet

A stage running multiple independent sub-pipelines over the same input documents in one pass.

Change Stream

A resumable, ordered stream of real-time data change events on a collection, database, or cluster.

Resume Token

An opaque token identifying a position in a change stream, enabling restart without event loss.

Atlas Search

Lucene-powered full-text search integrated into Atlas via the `$search` stage.

Atlas Vector Search

Approximate-nearest-neighbor (HNSW) vector index search via `$vectorSearch`, used for RAG and recommendations.

ASP

Atlas Stream Processing: managed continuous stream processing over Kafka/Change Streams sources with windowed aggregation.

CSFLE

Client-Side Field Level Encryption; fields are encrypted by the driver before leaving the application.

Queryable Encryption (QE)

MongoDB's in-use encryption allowing equality queries over encrypted fields without server-side decryption.

PITR

Point-In-Time Recovery: restoring a cluster to any second within the continuous-backup retention window using oplog replay.

Performance Advisor

Atlas feature that analyzes slow queries and recommends indexes.

8 Appendix B: Assessment & Grading Model

Each milestone capstone is graded on the following weighted rubric:

Category	Weight	What is assessed
Correctness	30%	Pipeline/system produces correct results end-to-end from a clean checkout.
Reliability & tests	20%	Automated data-quality tests, failing-case demo, failover/restore drills.
Security	15%	Least-privilege roles, secrets management, encryption, PII handling.
Cost awareness	15%	Monthly cost estimate with documented optimization decisions.
Documentation	10%	README quality, architecture diagram, runbook entries.
Demo	10%	Live walkthrough: deploy, break, observe, and recover the system.

Readiness score: the six capstone grades form 70% of the final readiness score. The Week 24 timed practice exam contributes the remaining 30%; a score of 75%+ on the practice exam plus all capstone rubrics satisfied indicates certification readiness.

9 Appendix C: Interview Preparation

1. Walk me through what happens, byte by byte, when a document is written to MongoDB. *(Week 1)*
2. When would you embed versus reference, and which anti-patterns have you seen in production? *(Weeks 2–3)*
3. How do you migrate a large PostgreSQL database to MongoDB with zero downtime? *(Week 4)*
4. Explain what can roll back during a primary failover and how write/read concerns prevent it. *(Week 5)*
5. How do you choose a shard key, and how do you fix a bad one on a live 20M-document collection? *(Weeks 6–7)*
6. How would you enforce GDPR data residency across a global cluster? *(Week 8)*
7. A query is slow: walk me through your `explain()` methodology and how the ESR rule guides index design. *(Weeks 9, 12)*
8. Replace five SQL JOINS with one aggregation pipeline — which stages do you use and why? *(Weeks 10–11)*
9. How do Change Streams and Atlas Stream Processing differ, and how do you make stream consumers idempotent? *(Weeks 15–16)*
10. Design the security architecture for a HIPAA-compliant MongoDB platform, including encryption in use. *(Weeks 17–19)*

10 Appendix D: Self-Check Answer Key

Week 1:

1. 60 seconds by default.
2. Committed writes made since the last checkpoint.
3. `wiredTiger.cache` (eviction and dirty-bytes counters).
4. The OS page cache needs RAM for file I/O buffering.
5. Latency stalls during checkpoints; tune cache size and disk I/O.

Week 2:

1. The Attribute Pattern (array of key/value attribute documents).
2. Groups many measurements per document, slashing metadata overhead.
3. When the working set is bloated by rarely-read fields.
4. Derived values, precomputed at write time.
5. Type 1 overwrites; Type 2 keeps versioned history with `valid_from/valid_to`.

Week 3:

1. When the “many” side is few and read together with the parent.
2. Unbounded document growth hits the 16MB limit and degrades writes.
3. Breaking: renaming a required field; non-breaking: adding an optional field.
4. Logs violations but still accepts the write.

5. The producer team owns it; enforced at ingestion, in CI, and in the database.

Week 4:

1. Writing to both systems; risk of divergence on partial failure.
2. To capture changes that occurred during the bulk load.
3. That both systems return equivalent results for live queries.
4. Retries and replays must not duplicate or corrupt data.
5. Keeping the source live and replicating back until cutover.

Week 5:

1. The new primary may lack writes the old primary acknowledged.
2. Capped replication log; secondaries may re-apply operations.
3. They vote without holding data, weakening durability semantics.
4. Acknowledgment only after the journal write is durable.
5. `readConcern: "majority"`.

Week 6:

1. `mongos` routes queries; the CSRS stores chunk and zone metadata.
2. Hash spreads evenly but defeats range targeting; range is the opposite.
3. All new writes land on the single highest chunk's shard.
4. A contiguous shard-key range; split when exceeding max chunk size.
5. Chunks between shards; copy cost plus latency on the donor shard.

Week 7:

1. High cardinality, low frequency, non-monotonic rate of change.
2. Adding suffix fields to the existing shard key live.
3. Every shard must be queried and results merged by `mongos`.
4. Few chunks limit distribution and parallelism across shards.
5. One shard shows disproportionate ops/CPU versus its peers.

Week 8:

1. A range of shard-key values (chunks).
2. Zone-shard EU keys to EU shards; verify with chunk distribution.
3. Routes reads to the member with the lowest network latency.
4. Directing reads to members tagged by hardware or region.
5. Minimizes cross-region latency and egress cost.

Week 9:

1. Equality fields first, then Sort, then Range.
2. All returned fields come from the index; no document FETCH.
3. At most one array (multikey) field per compound index.
4. When queries consistently target a small filtered subset.
5. Avoids blocking writes and cache pressure on the primary.

Week 10:

1. Reduces documents flowing into expensive later stages.
2. Runs multiple independent sub-pipelines over the same input.
3. Recursive traversal of hierarchies (e.g. org charts).
4. Outputs one document per array element.
5. Stages may spill to disk beyond 100MB RAM; 10–100x slower.

Week 11:

1. Window functions: ranks, moving averages, cumulative sums.
2. The pipeline aborts unless `allowDiskUse` is set.
3. `$rank` skips positions after ties; `$denseRank` does not.
4. The rate of change between window boundary documents.
5. A range window of -30 days to current over a date field.

Week 12:

1. Index scan versus full collection scan.
2. Profiled slow operations with timing and plan details.
3. Unused indexes that waste RAM and write throughput.
4. Queued read/write operations; the server is saturated.
5. An in-memory sort because no index provides the order.

Week 13:

1. Apache Lucene, integrated per Atlas cluster.
2. `$search` (or `$searchMeta`).
3. `autocomplete`.
4. Tokenizes and normalizes text (stemming, casing, stopwords).
5. Substring/partial matching on short fields like SKUs.

Week 14:

1. A dense vector capturing the semantic meaning of content.
2. Approximate nearest-neighbor graph; sub-linear lookup time.
3. BM25 keyword (`$search`) with vector (`$vectorSearch`) results.
4. When magnitudes vary; direction carries the meaning.
5. Retrieves semantically similar documents to ground the LLM prompt.

Week 15:

1. Ever-growing state exhausts memory/disk, slows checkpoints, and causes restarts.
2. Stream-collection joins: enriching each event with current Atlas data.
3. Events outside the window/allowed lateness miss matches or must be dropped.
4. Replay re-emits, but upserts on the dedup key make re-processing effect-free.
5. The token is invalidated; the consumer must re-bootstrap from a fresh snapshot.

Week 16:

1. A position in the stream enabling restart without event loss.
2. Crashes then reprocess at most the un-checkpointed events.
3. Pushing curated analytics back into operational tools (CRM).
4. Matching records to the same entity across systems' keys.
5. Freshness and immediacy versus API rate limits and cost.

Week 17:

1. x.509; no passwords to rotate, certificate-based identity.
2. Built-in roles are too broad for least-privilege access.
3. Adds collection/action privileges to an existing role.
4. Only the exact actions on the exact collections needed.
5. Signals misconfigured roles or an active probing attack.

Week 18:

1. CSFLE encrypts before the query; QE allows equality queries on ciphertext.
2. Deterministic enables equality matches; random prevents them.
3. The master key wraps DEKs stored in the key vault collection.
4. Equality (plus range/prefix in newer versions) only.
5. All encrypted data becomes permanently unrecoverable.

Week 19:

1. Private endpoints avoid public IPs and broad allowlists.
2. Strong transport encryption with controlled trust chains.
3. Which actions and collections generate audit events, reducing volume.
4. Tamper evidence and correlation across systems.
5. PrivateLink, TLS, authentication, RBAC, auditing.

Week 20:

1. Tiers cold data to S3/Blob as queryable Parquet.
2. Per-GB-scanned billing; partition pruning keeps costs low.
3. Live Atlas collections alongside archived cloud storage.
4. The window within which any point in time is restorable.
5. An untested backup is not a recovery strategy.

Week 21:

1. Projects, clusters, users, private endpoints, search indexes.
2. Prevents concurrent applies from corrupting shared state.
3. Reviews infrastructure diffs before they touch production.
4. CI secret stores or vaults; never in code.
5. Schema, index, and infrastructure changes deploy as one reviewed unit.

Week 22:

1. A Custom Resource Definition extending the Kubernetes API.

2. Provisioning, TLS rotation, upgrades, and scaling of clusters.
3. Replaces members one at a time, preserving availability.
4. A free local cluster with the same operator semantics.
5. The Operator reconciles K8s state; Ops Manager monitors and backs up.

Week 23:

1. Maximum data-loss window versus maximum time to restore service.
2. Nearest snapshot plus oplog replay to the target second.
3. Replication lag > 10s; connections > 80% of maximum.
4. Analyzes slow queries and recommends indexes.
5. A silently broken backup defeats the entire DR plan.

Week 24:

1. Replication, Consensus, and Sharding Mechanics (30%).
2. 75%+ under timed conditions.
3. 70%; the practice exam contributes 30%.
4. About 1.5 minutes per question; flag and revisit hard ones.
5. Trace the misconception to a domain and re-study it.

11 Appendix E: Datasets & Project Data

Public datasets

- **MongoDB Atlas sample datasets** — loadable free from the Atlas UI (“Load Sample Dataset”): `sample_mflix` (movies/comments; Weeks 9–13 aggregation and search), `sample_analytics` (customers/accounts/transactions; Milestone 3 analytics), and `sample_supplies` (orders with embedded items; Milestones 1–2 modeling and sharding). See <https://www.mongodb.com/docs/atlas/sample-data/>.
- **Olist Brazilian e-commerce dataset** (Kaggle: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>) — 9 relational CSVs (100k orders); ideal for the Milestone 1 relational-to-document migration capstone and the Milestone 4 recommendation engine.
- **MongoDB open datasets** (<https://github.com/mongodb/docs-assets> and <https://github.com/neelabalan/mongodb-sample-dataset>) — restaurants, airbnb, and other JSON corpora for indexing and aggregation practice.

Synthetic data generation

- **mgeneratejs** (<https://github.com/rueckstiess/mgeneratejs>) — template-driven document generation at volume (10M+ inserts in Week 1).
- **Python Faker** — realistic PII-style fields for capstone seed data.
- **Expected volumes per milestone:** Milestone 1: ~100k documents; Milestone 2: 20M documents (live shard-key refinement); Milestone 3: 100M documents; Milestones 4–6: 1–10M documents plus continuous Change Stream / Kafka event feeds.

12 Appendix F: Portfolio & Presentation Guide

Presenting each capstone on GitHub

Structure every capstone README as:

1. **Problem statement** and architecture diagram first — a reviewer should understand the system in 30 seconds.
2. **Setup instructions** that run end-to-end from a clean checkout (seed data scripts included).
3. **Screenshots/evidence**: dashboards, `explain()` plans, failover or PITR restore proof, alert configs.
4. **Cost estimate**: monthly Atlas footprint with the two optimization decisions you documented.
5. **Lessons learned** and known limitations — this section is what senior interviewers read first.

Resume bullet templates (two per milestone)

- **M1**: “Migrated a 15-table PostgreSQL schema to MongoDB using Relational Migrator with CDC, achieving zero-downtime cutover and a 5x query speedup.”
- **M1**: “Designed document models applying Attribute, Bucket, and Subset patterns, enforced with versioned `$jsonSchema` data contracts in CI.”
- **M2**: “Deployed a 4-shard multi-region MongoDB cluster with zone sharding enforcing GDPR data residency across 3 regions.”
- **M2**: “Refined shard keys live on 20M documents, reducing scatter-gather analytical queries by 90%.”
- **M3**: “Built aggregation pipelines (`$facet`, `$setWindowFields`, `$graphLookup`) over 100M documents served by fully covered ESR-compliant indexes.”
- **M3**: “Eliminated disk spilling and collection scans via `explain()`-driven tuning and `$indexStats` index pruning.”
- **M4**: “Implemented hybrid search combining Atlas Search (BM25) and Vector Search embeddings for a RAG recommendation engine.”
- **M4**: “Built an Atlas Stream Processing pipeline windowing Kafka events into real-time personalized recommendations.”
- **M5**: “Hardened a healthcare platform with Queryable Encryption via AWS KMS, x.509 auth, PrivateLink, and SIEM-integrated auditing.”
- **M5**: “Designed a 7-year data lifecycle with Atlas Online Archive tiering to Parquet while preserving federated queryability.”
- **M6**: “Automated multi-environment Atlas deployments with Terraform and GitOps CI/CD including schema-contract tests.”
- **M6**: “Operated MongoDB on Kubernetes via the Operator and validated DR with a quarterly PITR restore drill (RPO < 1 minute).”

Talking about your projects in interviews

Lead with the problem and the scale (document counts, throughput, regions), then quantify outcomes (latency reduction, cost saved, failover time). Pick one hard trade-off per project — shard key choice, write concern vs. latency, embedding vs. referencing — and explain what you rejected and why. Be ready to whiteboard the architecture diagram from memory and to defend a failure scenario: region outage, rolled-back writes, or schema drift caught in CI.

13 6-Month Progress Tracking Timetable

Week	Primary Focus	Key Project / Capstone	Status
1	WiredTiger Storage Engine	Cache & Eviction Tuning Benchmark	☐
2	Advanced Schema Patterns & Analytics Modeling	Attribute & Subset Pattern Refactoring + SCD2 History	☐
3	Modeling Relationships & Data Contracts	JSON Schema Validation Implementation	☐
4	Relational to MongoDB Migration	Cap 1: Relational Monolith Modernization	☐
5	Replica Set Internals	Failover & Election Simulation (rs.stepDown)	☐
6	Sharding Mechanics	Hash-based Sharded Cluster Deployment	☐
7	Cluster Balancing & Tuning	Live Shard Key Refinement	☐
8	Global Clusters & Multi-Region	Cap 2: Globally Distributed Sharded Cluster	☐
9	Index Mechanics & ESR Rule	Compound Index Execution Optimization	☐
10	Aggregation Framework Stage 1	Recursive \$graphLookup Pipeline	☐
11	Aggregation Framework Stage 2	Moving Averages with \$setWindowFields	☐
12	Performance Profiling & Tuning	Cap 3: Real-Time Analytical Engine	☐
13	Atlas Search (Lucene)	Custom Analyzer & Autocomplete Search	☐
14	Atlas Vector Search & GenAI	RAG Backend with Embeddings & Vector Index	☐
15	Atlas Stream Processing	Kafka Windowed Streaming Pipeline	☐
16	Change Streams, Eventing & Reverse ETL	Cap 4: Agentic E-Commerce Engine	☐
17	Authentication & RBAC	SCRAM + x.509 Custom Role Setup	☐
18	Encryption (CSFLE & QE) + GDPR Erasure	Queryable Encryption with AWS KMS	☐
19	Network Security & Auditing	Custom Audit Filters & Log Inspection	☐
20	Online Archiving	Cap 5: Hardened Healthcare Platform	☐
21	IaC & Zero-Downtime Schema Evolution	Atlas Terraform Provisioning Scripts	☐

Week	Primary Focus	Key Project / Capstone	Status
22	MongoDB Kubernetes Operator	Minikube Operator Deployment & Rolling Upgrade	☐
23	Backup, Observability & Disaster Recovery	Atlas PITR Restore Drill & Alert Configuration	☐
24	Certification Preparation	Cap 6: Production GitOps Deployment	☐

"In MongoDB, schema design dictates index strategy, index strategy dictates query execution, and sharding dictates global scale."

Best of luck on your 6-month journey to becoming a Master Data Engineer and Senior Architect in MongoDB!